buildings

MDPI

Article

Smart Building Recommendations with LLMs: A Semantic Comparison Approach

Ioannis Papaioannou [1,*], Christos Korkas [1,2,*] and Elias Kosmatopoulos [1,3]

[1] Centre for Research and Technology Hellas, Information Technologies Institute, 57001 Thessaloniki, Greece; kosmatop@iti.gr
[2] Electrical and Computer Engineering Department, University of Western Macedonia, 50100 Kozani, Greece
[3] Electrical and Computer Engineering Department, Democritus University of Thrace, 67100 Xanthi, Greece
* Correspondence: johnpap@iti.gr (I.P.); chriskorkas@iti.gr (C.K.); Tel.: +30-2310-464160 (C.K.)

Abstract

The increasing need for sustainable energy management in smart buildings calls for cost-effective solutions that balance energy efficiency and occupant comfort. This article presents a Large Language Model (LLM)-based recommendation system capable of generating proactive, context-aware suggestions from dynamic building conditions. The system was trained on a combination of real-world data and Sinergym simulations, capturing inputs such as weather conditions, forecasts, energy usage, electricity prices, and detailed zone parameters. Five models were fine-tuned and evaluated: GPT-2-Small, GPT-2-Medium, DeepSeek-R1-Distill-Qwen-1.5B, DeepSeek-R1-Distill-Qwen-7B, and GPT-4. To enhance evaluation precision, a novel metric, the Zone-Aware Semantic Reward (ZASR), was developed, combining Sentence-BERT with zone-level scoring and complemented by F1-Score metrics. While GPT-4 demonstrated strong performance with minimal data, its high inference cost limits scalability. In contrast, open-access models like DeepSeek-R1-Distill-Qwen-1.5B, DeepSeek-R1-Distill-Qwen-7B, and GPT-2-Medium required larger datasets but matched or exceeded GPT-4's performance at significantly lower cost. The system demonstrated adaptability across diverse building types, supported by heterogeneous datasets and parameter normalization. Importantly, the system was also deployed in a real-world multi-zone residential building in Thessaloniki, Greece. During a two-week operational period under near-identical weather and occupancy conditions, the model-assisted recommendations contributed to an estimated 10% reduction in electricity consumption, showcasing the practical potential of LLM-based recommendations in live building environments.

Keywords: smart buildings; Large Language Models (LLMs); recommendation systems; sentence BERT; GPT-2; GPT-4; DeepSeek

check for updates

Academic Editor: Antonio Caggiano

Received: 9 May 2025
Revised: 11 June 2025
Accepted: 23 June 2025
Published: 30 June 2025

Citation: Papaioannou, I.; Korkas, C.; Kosmatopoulos, E. Smart Building Recommendations with LLMs: A Semantic Comparison Approach. *Buildings* **2025**, *15*, 2303. https://doi.org/10.3390/buildings15132303

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Buildings contribute significantly to global energy consumption, highlighting the need for sustainable energy solutions. With rising concerns over environmental sustainability and operational efficiency, the implementation of innovative, data-driven approaches to energy management is crucial [1]. Smart buildings integrate automation technologies to optimize energy usage while maintaining occupant comfort, making energy efficiency a key factor in sustainable urban development.

Recent advancements in smart building technologies have focused on integrating Energy Management Systems (EMSs) with renewable energy sources, enhancing both

efficiency and user experience [2]. For instance, simulation-based EMS strategies optimize thermal comfort and energy costs in grid-connected photovoltaic (PV) microgrids by leveraging solar energy and occupancy schedules [3]. Additionally, Artificial Neural Networks (ANNs) and Fuzzy Logic have been implemented in HVAC systems to improve thermal comfort while reducing computational complexity, enabling deployment on embedded platforms [4]. Demand response algorithms utilizing closed-loop feedback mechanisms dynamically balance energy consumption and generation, reducing costs while preserving comfort [5].

Earlier works have also demonstrated the integration of distributed optimization frameworks for coordinating Energy Storage Systems (ESSs) and Electric Vehicles (EVs), yielding measurable cost reductions [6]. Complementary research has addressed anomaly detection in home appliance usage patterns for improving operational safety and energy awareness [7]. Reinforcement Learning (RL) algorithms, such as Self-Adapted Advantage-Weighted Actor–Critic, have shown superior performance over rule-based methods by balancing occupant comfort and energy efficiency [8]. While these approaches offer strong control capabilities, they often require specialized models or heuristics that limit flexibility across building configurations. Recent simulation environments such as Sinergym [9] have further supported the development and benchmarking of intelligent control strategies by offering realistic, EnergyPlus-based simulations tailored for Reinforcement Learning and smart building optimization tasks.

To overcome these challenges, the integration of Large Language Models (LLMs) into smart building automation presents a methodological advancement. As distinct from AI models tailored for specific, narrowly defined tasks, LLMs excel at processing diverse data streams, understanding contextual subtleties, and generating adaptive, human-like responses in real time. By embedding LLMs within EMSs, smart buildings can transition from rule-based automation to intelligent, self-learning systems capable of interpreting occupant preferences, external weather patterns, energy demand fluctuations, and operational constraints, all while making proactive, data-driven decisions [10]. This advancement ushers in a new era of autonomous, context-aware energy management, ensuring greater efficiency, cost-effectiveness, and occupant satisfaction. Recent advances further reinforce this trend. A range of approaches now demonstrate how LLMs can support smart building workflows. One line of research has shown that natural language descriptions of buildings can be automatically converted into energy simulation models through LLM-based platforms [11]. Follow-up studies have proposed prompt engineering techniques to guide LLM behavior in generating accurate building models, even without retraining [12]. Other works have introduced multimodal systems that combine vision and structured knowledge representations to produce personalized recommendations within indoor environments [13]. In parallel, the need for practical deployment of AI in building management has led to fast and hardware-efficient ML solutions, suitable for integration into real-world BMS infrastructure [14].

The incorporation of generative AI and language-based reasoning into energy management also raises a significant evaluation challenge. Traditional metrics such as energy savings, cost reductions, and comfort maintenance remain essential, but they are not sufficient to validate an LLM's contributions. It becomes crucial to ensure that the content generated by an LLM, whether it is a recommended action or an explanatory report, is semantically accurate and aligned with the objectives of energy efficiency. This introduces a new gap in the literature: how to systematically evaluate and trust AI outputs that operate in the language domain within the context of building management. Techniques developed in the NLP community can be repurposed to address this challenge. For instance, Sentence-BERT embeddings allow for semantic similarity comparisons between the

AI's recommendations and established best-practice actions [15]. Similarly, metrics like BERTScore [16] can quantitatively assess how closely a generated explanation matches an expert-written reference, ensuring alignment with domain-specific knowledge. Beyond automated evaluation, human-in-the-loop validation will play a crucial role: facility managers may initially need to review and approve AI-generated recommendations to build trust and prevent infeasible suggestions. Ultimately, ensuring the trustworthiness of an LLM-driven system will likely require a combination of semantic evaluation, constraint enforcement to prevent unsafe actions, and transparent reasoning mechanisms. This represents a fertile area for interdisciplinary research spanning AI, engineering, and human factors.

While LLMs are extremely promising, the present applications in energy management tend to be reactive rather than predictive. For example, GreenIFTTT [17] enables users to establish energy-efficient routines via conversational commands, while [18] demonstrates how LLMs adapt to user preferences without retraining. However, these approaches lack real-time environmental data integration, limiting their autonomous optimization capabilities. Other works focus on creative reasoning and control. For example, ref. [19] employs an LLM to interpret abstract commands like "make it cozy" but is limited to static tasks without environmental awareness. Similarly, ref. [20] applies GPT-4 for museum engagement, but its scope is confined to static environments rather than dynamic building operations. Additionally, GPT-4 HVAC optimization [21] has achieved notable energy savings but primarily targets short-term energy reduction, lacking adaptability to long-term environmental changes. Meanwhile, ref. [22] enhances privacy and secure data handling but does not provide real-time, context-aware recommendations. Although previous research explores LLMs in industrial IoT [23] and urban sustainability [24], their application in real-time building-level energy optimization remains underdeveloped. Moreover, while LLMs have been investigated for energy trading [25], their direct impact on autonomous, adaptive building operations has been overlooked.

2. Proposed Contribution

This article refines and extends Large Language Model (LLM)-based recommendation frameworks for smart building environments. The system was trained on datasets generated from real-world scenarios and Sinergym simulations, incorporating multi-level inputs such as real-time weather conditions, short-term forecasts, electricity prices, overall building energy use, and granular zone-level details including temperature, humidity, occupancy, and consumption. The overall methodology, illustrated in Figure 1, consists of data preprocessing, normalization of key parameters (e.g., energy consumption, photovoltaic production, and radiation), fine-tuning each LLM, and evaluating model outputs using both semantic and statistical metrics.

Five models were fine-tuned and evaluated to explore the trade-off between recommendation quality and computational cost: GPT-2-Small, GPT-2-Medium [26], DeepSeek-R1-Distill-1.5B, DeepSeek-R1-Distill-7B [27], and GPT-4 [28]. Evaluation was conducted using two key metrics: the Zone-Aware Semantic Reward (ZASR), a novel method developed in this article that extends Sentence-BERT to measure both sentence-level semantic similarity and zone-specific accuracy, and the F1-Score, which quantifies precision and recall across multi-zone outputs. This systematic evaluation enabled model performance to be assessed on varying scales and complexities, providing insights into accuracy, efficiency, and scalability trade-offs for application to smart buildings.

Compared to existing approaches, the contributions of this work can be summarized as follows:

- **Cost-Efficient, Scalable Recommendation System**: A smart building recommendation framework is proposed, based on fine-tuned Large Language Models (LLMs),

capable of operating in dynamic, heterogeneous environments with low computational demands and real-time adaptability.

- **Structured Model Comparison Across Scales**: A systematic comparison of five LLMs (GPT-2-Small, GPT-2-Medium, DeepSeek-R1-Distill-Qwen-1.5B, DeepSeek-R1-Distill-Qwen-7B, and GPT-4) was conducted, analyzing trade-offs between semantic recommendation quality, training effort, model complexity, and cost effectiveness. Notably, the system was also tested in a real-world multi-zone residential building in Thessaloniki, Greece, providing empirical insights into energy performance, model behavior, and deployment feasibility under operational conditions.
- **Zone-Aware Semantic Reward (ZASR) Evaluation**: A novel evaluation methodology, developed as part of this article, that decomposes each multi-zone recommendation into zone-specific segments. Each segment is individually compared with the expected output using Sentence-BERT, and the metric penalizes incorrect or unnecessary zone suggestions to reflect both semantic accuracy and spatial precision.
- **Diverse and Realistic Dataset Generation**: Training and testing datasets are created from both real world conditions and Sinergym simulations, capturing a broad range of operational, climatic, occupancy, and energy consumption scenarios to enhance model generalization and robustness.
- **Foundation for Future Full Automation**: Although current outputs are generated in natural language, the system architecture is designed to support direct integration with Building Management Systems (BMSs) for autonomous execution of proactive energy management strategies without human intervention.

Together, these contributions extend the existing work by offering a domain-adapted, scalable framework for deploying LLMs in smart building energy management.

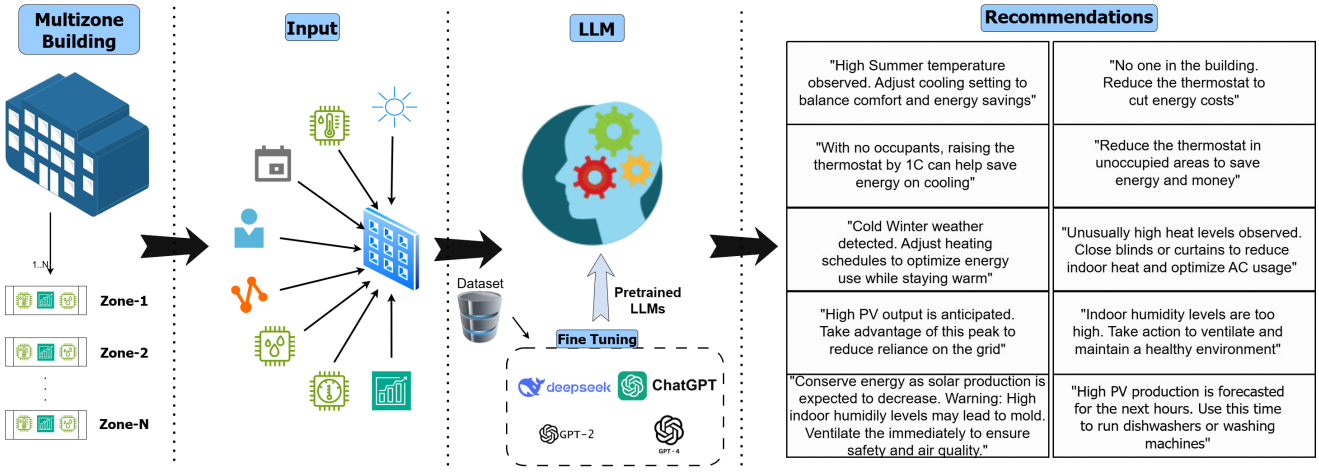


Figure 1. Methodology processing flow.

3. Control Strategy

The proposed control strategy leverages fine-tuned Large Language Models (LLMs) to generate proactive, context-aware recommendations that optimize energy efficiency and occupant comfort in smart buildings. By processing real-time environmental and operational data, the system dynamically adapts to evolving building conditions and external factors. Traditional energy management approaches often rely on static, rule-based logic, which may struggle to maintain performance across diverse and changing environments. In contrast, the proposed method employs advanced Natural Language Processing (NLP) to interpret complex data streams and generate flexible, human-readable recommendations that anticipate future building needs. The control framework continuously collects and

processes real-time sensor data, including temperature, humidity, solar radiation, occupancy, energy consumption, and predicted photovoltaic (PV) production, through the fine-tuned LLM. The model outputs actionable, zone-specific recommendations aligned with current and forecasted conditions. These outputs can be delivered directly to building occupants, facility managers, or integrated into automated Building Management Systems (BMSs) for execution, enabling seamless compatibility with existing control infrastructures. The operational logic emphasizes not only current sensor values but also predictive data (e.g., 3 h forecasts), enabling anticipatory adjustments that optimize both immediate and future building performance. The system's adaptability is further supported by parameter normalization, ensuring consistent performance across buildings of varying size and operational complexity. Overall, the control strategy demonstrates how LLMs can support real-time, adaptive energy management in complex, multi-zone building environments.

3.1. Data Integration

The success of the recommendation approach is dependent on the composition of various environmental, operational, and building-specific data sources. The input prompt provided to the language models is carefully structured to simulate the perspective of a smart building manager overseeing multiple zones, requiring proactive energy and comfort management. Environmental inputs include real-time ambient outside temperature, humidity, and solar radiation measurements and three-hour forecasts for temperature and solar radiation. Zone-specific parameters retain indoor conditions in each zone, such as temperature, humidity, occupancy, and real-time energy usage measurements. Operational information, such as the building's average energy consumption and current electricity price, is also incorporated to allow the system to weigh cost-optimized recommendations.

An example input format used during training and evaluation is as follows:

```
Instruction: You are a building manager of a 5-zone building.
The current weather conditions are:"
- Temperature: 23.5 °C"
- Humidity: 75%"
- Solar Radiation: 0.45"
The weather prediction for the next three hours:"
- Temperature: [23.0, 23.4, 23.1]"
- Solar Radiation: [0.42, 0.43, 0.48]"
Building energy usage:"
- Average zone consumption: 0.68"
- Current electricity price: €0.565/kWh"
Zone conditions:"
- Zone 1: Temp: 21.3°C, Humidity: 43%, Occupancy: Occupied, Consumption: 0.31"
- Zone 2: Temp: 19.2°C, Humidity: 46%, Occupancy: Occupied, Consumption: 0.52"
- Zone 3: Temp: 21.1°C, Humidity: 44%, Occupancy: Occupied, Consumption: 0.62"
- Zone 4: Temp: 22.1°C, Humidity: 49%, Occupancy: Empty, Consumption: 0.21"
- Zone 5: Temp: 20.4°C, Humidity: 52%, Occupancy: Occupied, Consumption: 0.27"
Instruction: Provide short, targeted recommendations for each zone that has unacceptable
values in temperature, humidity, or~energy consumption. Avoid mentioning zones that are
within normal thresholds. If~high or low solar radiation is forecasted, also include a
general building-level recommendation related to PV production and optimal appliance usage.
```

The structured input allows the models to reason over a combination of environmental factors, building conditions, occupancy patterns, and energy pricing to generate actionable recommendations. Zones operating within acceptable thresholds are intentionally omitted from responses to ensure spatially targeted and linguistically efficient outputs. To allow for consistency across various building geometries and energy consumption patterns, several factors such as solar radiation, energy consumption, and photovoltaic (PV) generation were normalized between 0 and 1 before model training. Normalization allows the models to be generalized to buildings of varying sizes and operational characteristics. Table 1

summarizes the key variables, their units, and the normalization ranges used throughout the control strategy.

Table 1. Input variables and value ranges.

Variable (Unit)	Value Range
Temperature_In (°C)	10 °C to 30 °C
Temperature_Out (°C)	−20 °C to 40 °C
Humidity_In (%)	0% to 90%
Humidity_Out (%)	0% to 100%
Thermostat Setpoint (°C)	16 °C to 28 °C
Electricity Price (€ per kWh)	0.0 to 2.5
Forecast_(1 h)_Temperature_Out (°C)	−10 °C to 40 °C
Forecast_(2 h)_Temperature_Out (°C)	−10 °C to 40 °C
Forecast_(3 h)_Temperature_Out (°C)	−10 °C to 40 °C
Forecast_(1 h)_Humidity_Out (%)	0% to 100%
Forecast_(2 h)_Humidity_Out (%)	0% to 100%
Forecast_(3 h)_Humidity_Out (%)	0% to 100%
Radiation (Solar + Direct) (W/m²)	0 to 1 (normalized)
Forecast_(1 h)_Radiation (W/m²)	0 to 1 (normalized)
Forecast_(2 h)_Radiation (W/m²)	0 to 1 (normalized)
Forecast_(3 h)_Radiation (W/m²)	0 to 1 (normalized)
Occupancy (Binary)	0: Unoccupied, 1: Occupied
Consumption (kWh)	0 to 1 (normalized)
PV_Production (kWh)	0 to 1 (normalized)
Forecast_(1 h)_PV_Production (kWh)	0 to 1 (normalized)
Forecast_(2 h)_PV_Production (kWh)	0 to 1 (normalized)
Forecast_(3 h)_PV_Production (kWh)	0 to 1 (normalized)

3.2. Dataset Creation and Preprocessing

To enable the system to be adapted and generalized effectively, 11 categories of recommendations were developed to address different scenarios related to energy efficiency and thermal comfort in smart buildings. Some of these include optimizing energy use during high or low photovoltaic (PV) production, managing thermostat settings based on occupancy, and addressing indoor air quality concerns, such as elevated humidity levels. These suggestions and the circumstances that lead to them are compiled in Table 2. For example, forecasts of high PV production encourage the usage of energy-intensive appliances, whereas forecasts of low PV production lead to conservation actions. Ventilation is recommended in situations of excessive humidity, and thermostat adjustments are recommended for vacant buildings to lower energy demands. By adding an "All zones are within acceptable limits" category, the system avoids unnecessary interventions under optimal conditions, enhancing overall robustness and ensuring recommendations are issued only when needed.

For each type of suggestion, 20 linguistic variants were created for each recommendation to ensure lexical diversity and prevent overfitting. These variations make sure the model produces dynamic and context-aware recommendations by expressing the same

information in multiple ways. To ensure wide generalizability, the dataset was generated to comprehensively cover all possible combinations of input parameters, including temperature, humidity, occupancy, energy consumption levels, and PV production forecasts, both per zone and at building level. The dataset was created utilizing a variety of climatic circumstances and real-world meteorological data. A substantial portion of the dataset was enriched with data collected from a real, multi-zone smart building located in Thessaloniki, Greece, which allowed the model to learn from realistic usage patterns, consumption behavior, and environmental conditions. By integrating a broad range of triggers, scenarios, and message modifications, the model is trained to generate natural language outputs with high flexibility and robustness, appropriate for diverse real-world applications.

Table 2. Recommendation types and trigger conditions.

ID	Recommendation	Trigger Condition
1	High PV production forecast. Use energy-intensive appliances.	High radiation forecast for the next hours.
2	Low PV production forecast. Limit energy-intensive usage.	Low radiation forecast for the next hours.
3	Lower thermostat as the building is unoccupied and the epoch is winter.	Unoccupied, low temperature.
4	Raise thermostat as the building is unoccupied and the epoch is summer.	Unoccupied, high temperature.
5	High humidity detected in the building. Ensure proper ventilation.	High indoor humidity levels detected.
6	High energy consumption in an unoccupied building. Check systems.	Unoccupied, high energy use.
7	All zones are within acceptable limits.	Optimal conditions.
8	High PV production forecast and high humidity detected in the building.	High radiation and high indoor humidity.
9	Low PV production forecast and high humidity detected in the building.	Low radiation and high indoor humidity.
10	Very high outdoor temperatures. Optimize AC usage.	High outdoor temperatures in summer.
11	Very low outdoor temperatures. Optimize heating usage.	Low outdoor temperatures in winter.

3.3. System Specifications for Model Training

The fine-tuning of the GPT-2-Small, GPT-2-Medium, DeepSeek-R1-Distill-Qwen-1.5B, and DeepSeek-R1-Distill-Qwen-7B models was conducted on a high-performance system equipped with an x86_64-based CPU, an NVIDIA GeForce RTX 3090 GPU, and 125.7 GB of RAM, providing the computational resources necessary for large-scale model training and evaluation. The system operated under Linux 6.8.0-49-generic and employed PyTorch 2.5.1 with CUDA 12.4 for optimized GPU acceleration. Leveraging the parallel processing capabilities of the NVIDIA RTX 3090 significantly reduced training times, enabling efficient experimentation across different model architectures. GPT-4 evaluations were performed via API-based interactions without local fine-tuning, consistent with OpenAI's usage policies. The detailed system specifications are presented in Table 3.

Table 3. System specifications for model fine-tuning.

Component	Specification
CPU	x86_64-based
GPU	NVIDIA GeForce RTX 3090
RAM	125.7 GB
Operating System	Linux 6.8.0-49-generic
Framework	PyTorch 2.5.1 with CUDA 12.4

4. Experimental Results

This section presents the evaluation methodology and comparative results for the language models used in this article. The experimental setup for each model is outlined, includ-

ing training configurations, model sizes, and fine-tuning strategies, ensuring a transparent basis for comparison across architectures of varying scale and complexity. The proposed **Zone-Aware Semantic Reward (ZASR)**, a novel evaluation metric specifically designed for smart building applications, is then introduced. ZASR extends traditional BERT-based similarity scoring by incorporating spatial alignment between expected and generated zone-level recommendations, allowing simultaneous assessment of semantic accuracy and spatial correctness. Alongside ZASR, F1-Score metrics are reported, to capture precision and recall across multi-zone outputs. The comparative analysis covers GPT-2 (Small, Medium), DeepSeek-R1-Distill-Qwen (1.5 B, 7 B), and GPT-4 models. The findings reveal that while GPT-4 achieves excellent semantic alignment with minimal fine-tuning, its high inference cost limits its practicality for large-scale or real-time deployment. In contrast, the DeepSeek models deliver comparable or superior performance at lower computational expense, while GPT-2 models, although lightweight and fast, struggle to generalize and generate novel, contextually adaptive text. These results highlight critical trade-offs between model size, output quality, generalization ability, and cost, offering practitioners clear guidance for selecting models that align with their deployment priorities and operational constraints.

4.1. Experimental Setup

To benchmark the performance of smaller-scale autoregressive models, the **GPT-2** model was fine-tuned on instruction–response pairs that describe the state of the building and its recommended operational adjustments. Each example was formatted as a single line of plain text by concatenating the instruction and response, then tokenized using the GPT-2 tokenizer. A maximum sequence length of 589 tokens was applied, matching the length of the longest sample in the dataset, ensuring all samples were fully preserved during training. The model was fine-tuned using the HuggingFace `Trainer` API. The final training configuration consisted of a per-device batch size of 4, a total of 10 epochs, and evaluation every 200 steps. Checkpoints and logs were also generated every 200 steps. The standard `DataCollatorForLanguageModeling` with masked language modeling disabled (`mlm = False`) was used, following a causal language modeling setup with cross-entropy loss. All the weights of the GPT-2 model were updated during training, without applying parameter-efficient tuning techniques such as LoRA. The model and tokenizer were saved at the end of training, along with logs and intermediate metrics. In addition to this configuration, several experiments were conducted, exploring different batch sizes, dataset scales, and both the `GPT2-Small` (124 M parameters) and `GPT2-Medium` (345 M parameters) variants, aiming to assess trade-offs between model size, performance, and training cost. These experiments and comparisons are presented in Section 4.4.

To assess the capacity of Large Language Models in producing zone-specific building recommendations, the **DeepSeek-R1-Distill-Qwen-1.5B** and the **DeepSeek-R1-Distill-Qwen-7B** models were fine-tuned using instruction–response text pairs. Each input was preprocessed into a structured format consisting of an instruction describing the environmental and operational context, followed by the expected response, using the following template:

```
Instruction: <building state>\nResponse: <recommended actions>
```

Low-Rank Adaptation (LoRA) was employed to enable parameter-efficient fine-tuning without modifying the full parameter space of the base model. The configuration included a rank $r = 8$, scaling factor $\alpha = 16$, dropout rate of 0.05, and no bias adaptation. LoRA adapters were applied to the attention projection layers (`q_proj`, `k_proj`, `v_proj`, and `o_proj`). The model was loaded in 4-bit precision using a quantization-aware configuration to minimize memory and computation requirements. A maximum sequence length of 589 tokens was selected during tokenization, corresponding to the longest input sequence observed in the dataset, to avoid truncation and preserve complete contextual

information. The tokenizer was configured to pad all sequences to this fixed length. Fine-tuning was performed using the HuggingFace `Trainer` API with a per-device batch size of 2, gradient accumulation over 4 steps, a learning rate of 2×10^{-5}, and a total of 5 training epochs. Mixed-precision (FP16) training was enabled to accelerate convergence. The training objective followed a causal language modeling formulation without masked language modeling, using the standard cross-entropy loss.

Finally, to benchmark against a state-of-the-art large language model, **GPT-4-o** (August 2024 release) using OpenAI's supervised fine-tuning API. The dataset consisted of 120 prompt–response pairs, each describing building environmental conditions and the corresponding recommended actions. The model was fine-tuned for 5 epochs with a batch size of 4, using a total of 160,715 trained tokens. Default API parameters were applied, including a learning rate multiplier of 1. The base model used was `gpt-4o-2024-08-06`, and the fine-tuned model checkpoint was saved and evaluated via the OpenAI API.

4.2. Zone-Aware Semantic Reward for Fine-Grained Evaluation

Evaluating the quality of recommendations in smart building systems requires not only assessing semantic similarity but also ensuring that the correct spatial context, namely, the zone associated with each recommendation, is preserved. A mismatch between zone attribution, even when textual similarity is high, can lead to significant operational errors within a building. To address this limitation, a Zone-Aware Semantic Reward (ZASR) evaluation methodology was developed, which measures the quality of generated outputs by simultaneously considering semantic accuracy, zone correctness, and the presence of extraneous or missing information.

The ZASR evaluation process consists of four main stages:

- **Zone Extraction**: Both the expected and the generated recommendations are parsed to extract zone-specific messages using regular expressions.
- **Sentence Splitting**: The extracted messages are split into individual sentences based on punctuation rules, ensuring high accuracy even when periods are missing spaces.
- **Sentence-Level Matching**: For each expected zone message, sentences are compared individually against predicted sentences using Sentence-BERT embeddings. Each expected sentence is matched with the most semantically similar predicted sentence, and the corresponding similarity score is recorded.
- **Reward and Penalty Calculation**: The final reward is based on the mean semantic similarity across all matched sentences, adjusted with penalties for structural errors, as described below.

Following extraction and sentence-level decomposition, each expected sentence is paired with its closest predicted sentence within the corresponding zone. Semantic similarity is computed using Sentence-BERT embeddings, and only similarity scores above a minimum threshold (e.g., 0.40) are collected. Letting S_i denote the similarity of the i-th matched sentence and E be the total number of expected sentences, the raw score is calculated as the average:

$$\text{Raw Score} = \frac{1}{E} \sum_{i=1}^{E} S_i \tag{1}$$

This formulation allows the reward to reflect not just whether a match exists, but also the quality of the match, with higher similarity values contributing more positively. In addition to semantic quality, ZASR penalizes structural inconsistencies. Predicted sentences that are unmatched or zones that are extraneous (i.e., present in the prediction but not in the expected output) incur penalties. The penalty factor, combining penalties for extra unmatched sentences S_{extra} and extra zones Z_{extra}, is given by

$$\text{Penalty Factor} = w_z \cdot Z_{\text{extra}} + w_s \cdot S_{\text{extra}} \quad (2)$$

where w_z and w_s are tunable weights controlling the severity of each type of penalty.

The final penalized score is then computed as

$$\text{Penalized Score} = \text{Raw Score} \times (1 - \text{Penalty Factor}), \quad (3)$$

ensuring that excessive mismatches or over-generation appropriately lower the reward. The penalized score is normalized between 0 and 1 to avoid negative values, and the final scaled reward is obtained by multiplying by a scaling factor $S_{\max}$, typically set to 100:

$$\text{ZASR Score} = S_{\max} \times \max(0, \min(1, \text{Penalized Score})) \quad (4)$$

For cases where the expected recommendation does not contain any zone-specific messages but the generated output introduces one or more zones, ZASR applies a penalty proportional to the number of extraneous zones, ensuring that models are discouraged from generating unnecessary zone-level suggestions. In cases where neither the expected nor the predicted recommendations explicitly mention zones, ZASR computes the semantic similarity between the full texts, with the similarity value directly determining the reward. To assess and compare the semantic performance of different language models under this updated evaluation scheme, a test set of 165 prompts was used. Each model was evaluated across all 165 cases, and the mean ZASR score was computed to represent its final semantic performance. The results are summarized in Section 4.4, covering model sizes, inference times, and ZASR scores.

4.3. Training Time Efficiency Analysis

Table 4 presents the training time comparison for GPT-2 models using both CPU and GPU resources, highlighting the impact of hardware type, model size, and dataset scale on training time efficiency. GPU-based training consistently outperformed CPU-based training, reducing computational time by up to 28 times. For example, training the GPT-2-Medium model on a dataset of 119,700 samples required 458.36 h on a CPU but only 18.56 h on a GPU. Similarly, for the same dataset the GPT-2-Small model completed training in 158.65 h on CPU versus 5.45 h on GPU. Scaling up the dataset to 239,400 samples further accentuated this trend. The GPT-2-Small model required 301.34 h on CPU and 11.32 h on GPU, while the GPT-2-Medium model took 920.56 h on CPU and 33.13 h on GPU. Across all configurations, the GPT-2-Small variant consistently trained faster, offering a favorable trade-off between computational efficiency and adaptability. This makes the small model particularly attractive for scalable smart building energy management applications, where rapid retraining or fine-tuning is often required.

Table 4. Training time comparison for GPT-2 (CPU and GPU).

Dataset_Size	Epochs	Batchsize	GPT-2	CPU (h)	GPU (h)
119,700	10	4	small	158.65	5.45
119,700	10	4	medium	458.36	18.56
239,400	10	4	small	301.34	11.32
239,400	10	4	medium	920.56	33.13

In addition to the GPT-2 training comparisons presented in Table 4, Table 5 extends the analysis by including larger and more recent models such as DeepSeek-R1-Distill-Qwen-1.5B, DeepSeek-R1-Distill-Qwen-7B, and GPT-4. This table highlights how model size and

architecture complexity influence training time, even when using the same dataset size, number of epochs, and batch size where possible. As expected, the larger models require significantly longer training times. For instance, the DeepSeek-R1-Distill-Qwen-7B model, despite using a batch size of 2, required 164 h to train 119,700 samples over 10 epochs, and 256 h for 239,400 samples. In contrast, GPT-2-Small trained the same dataset in just 5.45 and 18.56 h, respectively. The DeepSeek-R1-Distill-Qwen-1.5B model showed a more moderate increase in training time, requiring 29 to 56 h for 239,400 samples, demonstrating the scalability benefits of distilled architectures relative to their parameter size.

Notably, GPT-4, accessed via API rather than local training, completed fine-tuning on a small dataset of 120 samples in approximately 0.18 h (around 11 min). While GPT-4's training time was not directly comparable to local training times, due to differences in infrastructure and processing pipelines, it demonstrated the potential speed of commercial LLM fine-tuning for small-scale tasks. Overall, Table 5 confirms that although newer and larger models like DeepSeek and GPT-4 provide advanced language capabilities, their training efficiency heavily depends on computational resources and dataset scale. For many practical smart building applications, smaller models such as GPT-2-Small offer a compelling trade-off between performance and computational cost, particularly when frequent retraining is needed.

Table 5. Comparison of training time across Large Language Models (LLMs).

Model	Dataset_Size	Epochs	Batchsize	Time (h)
GPT-2-Small	119,700	10	4	5.45
	239,400	10	4	11.32
GPT-2-Medium	119,700	10	4	18.56
	239,400	10	4	33.16
DeepSeek-R1-Distill-Qwen-1.5B	119,700	10	2	29.35
	119,700	10	4	23.58
	239,400	10	2	55.45
	239,400	10	4	46.85
DeepSeek-R1-Distill-Qwen-7B	119,700	10	2	164.54
	239,400	10	2	256.76
GPT-4	120	5	4	0.18

4.4. Model Performance Under Zone-Aware Semantic Reward Evaluation

Table 6 presents a comprehensive comparison of the five evaluated language models, detailing their size (number of parameters), average inference time per prompt (seconds), semantic performance as measured by the Zone-Aware Semantic Reward (ZASR), and F1-Score. This dual-metric evaluation provides a deeper insight into both the linguistic and operational accuracy of the generated recommendations. GPT-2-Medium and DeepSeek-R1-Distill-Qwen-7B achieved the highest ZASR scores (82.24% and 80.9%, respectively), indicating strong alignment between the models' outputs and the expected zone-specific recommendations. Notably, GPT-2-Small, the lightest model evaluated (124M parameters), achieved a commendable ZASR of 75.4%, demonstrating its ability to deliver acceptable performance under strict resource constraints. DeepSeek-R1-Distill-Qwen-1.5B also performed competitively (80.2%), confirming its efficiency as a mid-range option. While GPT-4 delivered strong performance (79.11% ZASR), these results highlight the potential of smaller, open-access models to deliver high-quality recommendations when adequately fine-tuned.

In terms of F1-Score, DeepSeek-R1-Distill-Qwen-7B led with 0.912, closely followed by GPT-2-Medium (0.901) and DeepSeek-R1-Distill-Qwen-1.5B (0.882), indicating that these models not only generate semantically accurate recommendations but also maintain precision and recall across multiple test cases. GPT-2-Small achieved an F1-Score of 0.78, and GPT-4 registered 0.84, again highlighting the balance between model size and

performance. Inference times ranged from 2 to 3.1 s, with GPT-2 variants offering the fastest responses (2 s), while DeepSeek-R1-Distill-Qwen-7B required 3.1 s, due to its larger architecture. GPT-4 maintained a competitive inference time of 2.4 s, benefiting from its optimized infrastructure despite its large size. These results underscore the trade-offs between model complexity, computational demand, and performance. While larger models like DeepSeek-R1-Distill-Qwen-7B deliver top-tier accuracy, smaller models such as GPT-2-Medium and GPT-2-Small offer compelling alternatives when computational resources or latency constraints are critical factors.

Table 6. Model comparison based on parameters, inference time, and ZASR Score.

Model	Dataset Size	Number of Parameters	Inference Time (s)	ZASR Score (%)	F1-Score
GPT-2 Small	119,700	124 M	2.1	75.4	0.78
GPT-2 Medium	119,700	345 M	2.0	82.2	0.90
DeepSeek-R1-Distill-Qwen-1.5B	119,700	1.5 B	2.3	80.2	0.88
DeepSeek-R1-Distill-Qwen-7B	119,700	7 B	3.1	80.9	0.91
GPT-4	120	175 B	2.4	79.1	0.84

4.5. Feed-Forward Evaluation and Practical Insights

The system's evaluation involved feed-forward inference to test the models' ability to generate actionable and context-aware recommendations from structured prompts. As shown in Figure 2, each prompt included detailed environmental data (e.g., temperature, humidity, solar radiation), energy metrics (e.g., consumption, electricity price), and zone-specific parameters (occupancy, consumption per zone). These inputs simulated a real-world scenario for a five-zone building and instructed the model to provide targeted energy management advice. The five models fine-tuned and evaluated were GPT-2-Small, GPT-2-Medium, DeepSeek-R1-Distill-Qwen-1.5B, DeepSeek-R1-Distill-Qwen-7B, and GPT-4. To illustrate the differences in recommendation quality, the following images present side-by-side examples of outputs generated by each model in response to the same prompt.

```
Instruction: You are a building manager of a 5-zone building. The current weather conditions are:
- Temperature: 23.5°C
- Humidity: 54%
- Solar Radiation: 0.45

The weather prediction for the next three hours:
- Temperature: [23.0, 23.4, 23.1]
- Solar Radiation: [0.42, 0.43, 0.48]

Building energy usage:
- Average zone consumption: 0.68
- Current electricity price: €0.565/kWh

Zone conditions:
- Zone 1: Temp: 19.8°C, Humidity: 93%, Occupancy: Occupied, Consumption: 0.31
- Zone 2: Temp: 20.1°C, Humidity: 53%, Occupancy: Occupied, Consumption: 0.26
- Zone 3: Temp: 22.5°C, Humidity: 89%, Occupancy: Occupied, Consumption: 0.31
- Zone 4: Temp: 19.2°C, Humidity: 86%, Occupancy: Occupied, Consumption: 0.82
- Zone 5: Temp: 19.2°C, Humidity: 86%, Occupancy: Occupied, Consumption: 0.82

Instruction: Provide short, targeted recommendations for each zone that has unacceptable values
in temperature, humidity, or energy consumption. Avoid mentioning zones that are within normal thresholds.
If high or low solar radiation is forecasted, also include a general building-level recommendation
related to PV production and optimal appliance usage.
Response:
```

Figure 2. Structured prompt example for smart building recommendation task.

Figure 3 shows the output generated by the GPT-2-Medium model, which provides targeted, zone-specific recommendations primarily focused on humidity control. While GPT-2-Medium balances simplicity and practicality, its outputs tend to follow repetitive phrasings and show limited ability to generate novel or varied sentence structures compared to larger models. Notably, in this example, the model failed to detect and address the high energy consumption observed in Zone 5, underscoring its difficulty in capturing multiple simultaneous operational issues.

```
Generated Output:

Zone 1: Is experiencing severe humidity levels — activate air treatment systems.
 Zone 3: Is experiencing severe humidity levels — activate air treatment systems.
 Zone 4: Is overly humid — turn on the dehumidifier and ventilate the space. Shows high power usage
 Zone 5: Has excessive moisture — use dehumidifier and increase airflow.
```

Figure 3. GPT-2-Medium fine-tuned generated output.

In addition, Figure 4 provides a detailed example of the output generated specifically by the DeepSeek-R1-Distill-Qwen-Qwen-7B model for the same test case. This output highlights how the model produces zone-specific recommendations, addressing both humidity and energy consumption issues with precise, actionable advice. For instance, it suggests turning on dehumidifiers, increasing airflow, and reducing unnecessary loads in zones with high humidity and excessive power usage. Notably, DeepSeek-R1-Distill-Qwen-7B demonstrated a strong capacity to generate novel, contextually appropriate phrasing, avoiding the repetitive patterns observed in GPT-2-Medium. This reflects the model's advanced ability to interpret multiple input parameters simultaneously and deliver diverse, targeted interventions for each zone, resulting in significantly improved recommendation quality.

```
Generated Output:

Zone 1: Is overly humid — turn on the dehumidifier and ventilate the space.
 Zone 3: Has excessive moisture — use dehumidifier and increase airflow.
 Zone 4: Shows dangerously high humidity — take immediate action with ventilation.
 Shows high power usage — reduce unnecessary loads in this zone.
 Zone 5: Has excessive moisture — use dehumidifier and increase airflow. Is above the energy limit —
 take corrective action to lower consumption. Requires targeted maintenance to improve efficiency.
```

Figure 4. DeepSeek-R1-Distill-Qwen-7B fine-tuned generated output.

Furthermore, Figure 5 showcases the output generated by the GPT-4 model for the same scenario. GPT-4's recommendations included both a building-level suggestion, highlighting the opportunity to optimize appliance usage during high PV production, and precise, zone-specific advice for each affected area. Notably, GPT-4 maintained clear and structured language, recommending dehumidifier activation in all critically humid zones and monitoring of energy consumption where high usage is detected. This output reflects GPT-4's ability to balance clarity and operational relevance while addressing both building-level and zone-specific needs.

Overall, the feed-forward evaluation highlights clear differences in the behavior and technical capabilities of the models. GPT-2-Medium, despite its smaller size, delivered concise and actionable recommendations but showed limited linguistic diversity and occasional difficulty in simultaneously addressing multiple operational issues. DeepSeek-R1-Distill-Qwen-7B demonstrated a strong ability to integrate multiple input factors and generate diverse, contextually nuanced outputs, outperforming the smaller models in both precision and adaptability. GPT-4 maintained highly structured, comprehensive recommendations, balancing building-level and zone-specific advice; however, its advanced generative capabilities come with higher computational and financial costs, as discussed in Section 4.7.

These results underscore the importance of balancing model complexity, accuracy, and operational cost when selecting language models for scalable smart building applications.

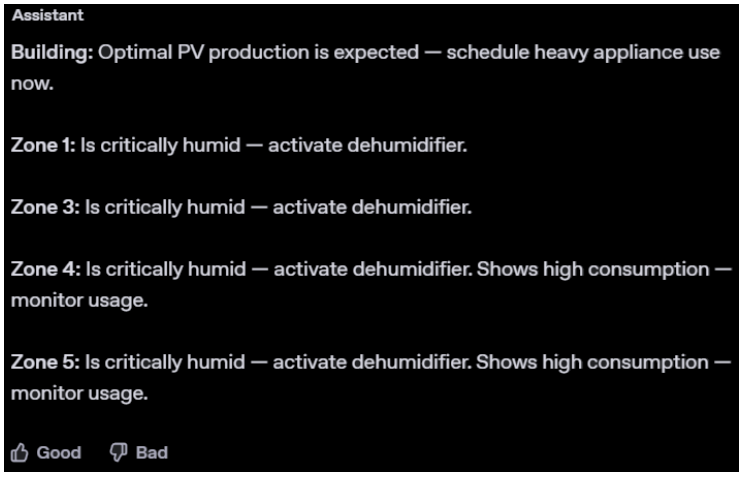


Figure 5. GPT-4 fine-tuned generated output.

To further support the qualitative observations presented above, the Zone-Aware Semantic Reward (ZASR) score was computed for each model's output in response to the same structured prompt. The results were as follows: GPT-2-Medium, 0.67; DeepSeek-R1-Distill-Qwen-1.5B, 0.81; and GPT-4, 0.85. These scores reflect each model's ability to correctly target only those zones with unacceptable conditions (e.g., high humidity or energy consumption) while maintaining semantic coherence. The comparison is summarized in Table 7. Notably, all the models correctly ignored Zone 2, which had no violations, demonstrating consistent alignment with instruction-based filtering. The ZASR metric thus provides a quantitative layer of validation to complement the feed-forward examples and ensures that the presented outputs are not anecdotal but systematically evaluated.

Table 7. ZASR evaluation of model outputs on the same prompt.

Model	ZASR Score	Distinct Zone Matches	Comment
GPT-2-Medium	0.67	Zones 1, 3, 4, 5	Less specific wording; no general building suggestion.
DeepSeek-R1-Distill-Qwen-1.5B	0.81	Zones 1, 3, 4, 5	More detailed instructions; clear separation of issues.
GPT-4	0.85	Zones 1, 3, 4, 5	Included building-level suggestion; concise phrasing.

4.6. Real-World Deployment and Impact in a Multi-Zone Smart Building in Thessaloniki of Greece

To demonstrate the real-world applicability of the proposed recommendation engine, a deployment scenario is presented based on a pilot smart building environment currently used for system evaluation. The building illustrated in Figure 6 is a 500 m^2 three-story mixed-use facility located in Thessaloniki, Greece, combining residential and commercial functions. It is equipped with rooftop photovoltaic (PV) panels, one heat pump per floor, and a comprehensive IoT infrastructure. Each floor operates as a distinct thermal zone, resulting in three total zones: Ground Floor (Zone 1), First Floor (Zone 2), and Second Floor (Zone 3).

Environmental and operational data, temperature, humidity, CO_2 levels, occupancy, and electricity consumption were collected using environmental sensors and smart meters. These data streams were aggregated at the edge and transmitted over low-latency communication protocols (Z-Wave and 5G). Input vectors were constructed every 15 min, combining

current zone-level readings, PV production, electricity pricing, and three-hour weather forecasts. The recommendation engine interfaced with the building systems through a RESTful API, returning textual instructions for actionable changes.

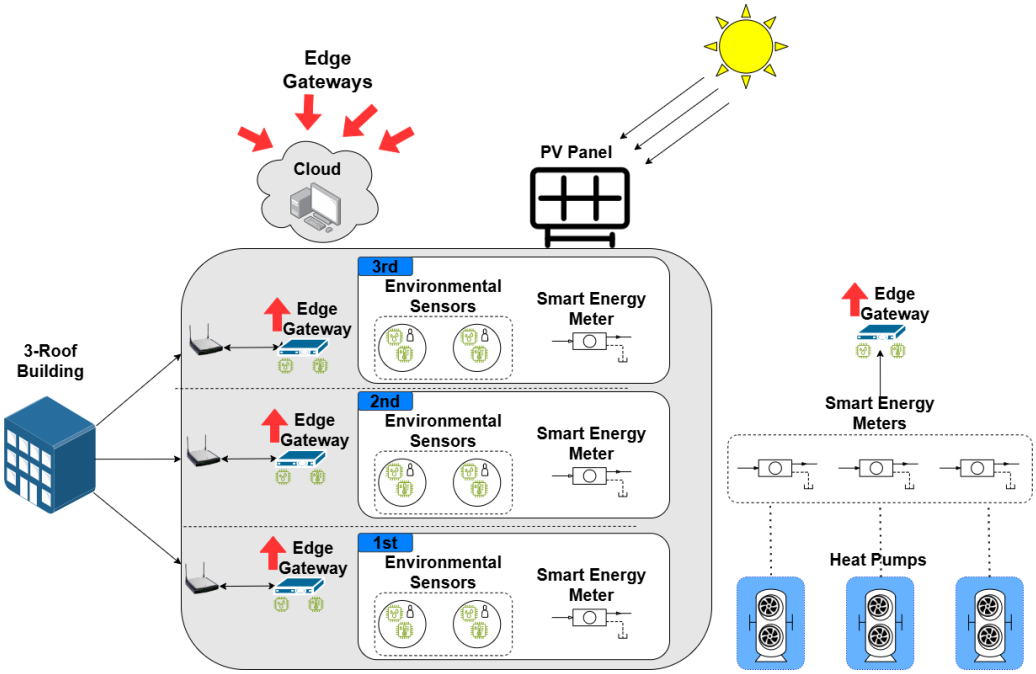


Figure 6. Real case in Thessaloniki pilot.

To evaluate real-time suitability, inference latency tests were conducted for each LLM model using a local deployment on an NVIDIA RTX 3060 GPU and the GPT-4 API. The results are summarized as follows:

- GPT-2-Small: 1.7 s;
- GPT-2-Medium: 2.1 s;
- DeepSeek-R1-Distill-Qwen-1.5B: 2.3 s;
- DeepSeek-R1-Distill-Qwen-7B: 3.1 s;
- GPT-4 (API): 2.4 s.

All the models, except GPT-4, achieved response times well within the 15-min control loop window, confirming their viability for integration into near-real-time decision pipelines. DeepSeek-R1-Distill-Qwen-1.5B was selected for full deployment, due to its favorable trade-off between latency, semantic performance, and cost-efficiency.

The live evaluation was conducted over a two-week period. From 18 May to 25 May, the building followed default HVAC schedules and manual control by occupants (baseline period with no LLM model applied). From 26 May to 2 June, the LLM engine (DeepSeek-R1-Distill-Qwen-1.5B) was activated, delivering recommendations every 15 min using the same environmental inputs. These dates correspond directly to the x-axis of Figures 7–9, marking the transition from the baseline week to the intervention week with LLM control. These suggestions guided occupants or automated systems to adjust temperature, ventilation, and appliance usage based on real-time zone conditions and PV forecasts.

As illustrated in Figures 7–9, energy consumption decreased consistently across all three zones during Week 2, where the LLM-generated recommendations were applied: Zone 1 showed a noticeable reduction in peak heating activity, especially during morning hours. Zone 2 exhibited smoother heating curves with lower energy spikes, indicating improved control based on forecasted comfort conditions. Zone 3 demonstrated the clearest reduction in total energy use, with overall consumption flattened during both daytime

and night-time hours. Cumulatively, this resulted in a 9.96% overall reduction in energy consumption across the entire building. The observed savings are attributable to LLM-driven instructions that reduced HVAC activity in already-comfortable or unoccupied rooms, shifted usage to periods of higher PV production, and optimized HVAC cycling behavior. These findings confirm that the model not only generates semantically correct suggestions but also translates them into measurable, zone-specific energy efficiency gains in an operational building context.

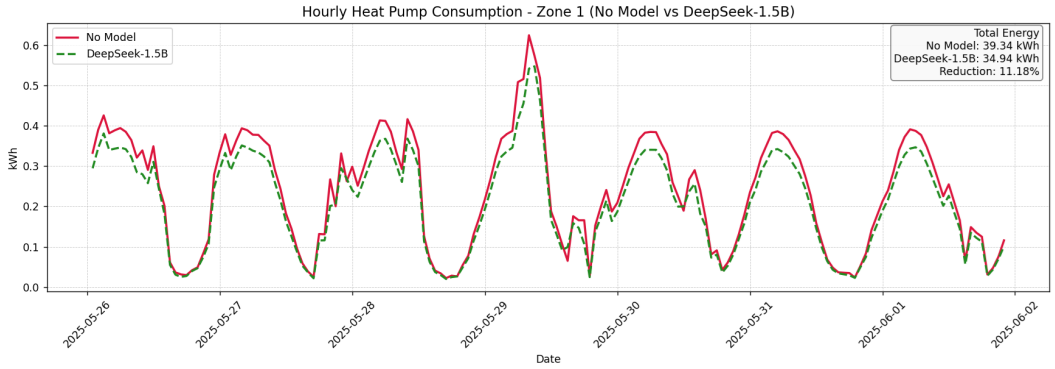


Figure 7. Zone 1 Thessaloniki pilot.

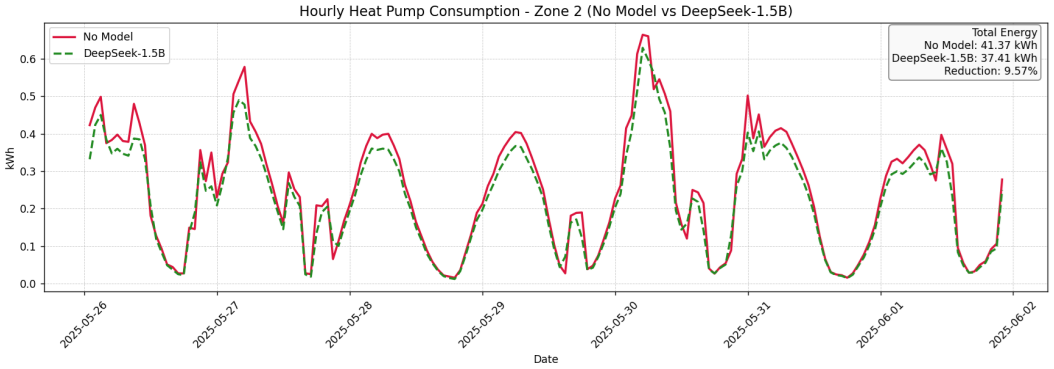


Figure 8. Zone 2 Thessaloniki pilot.

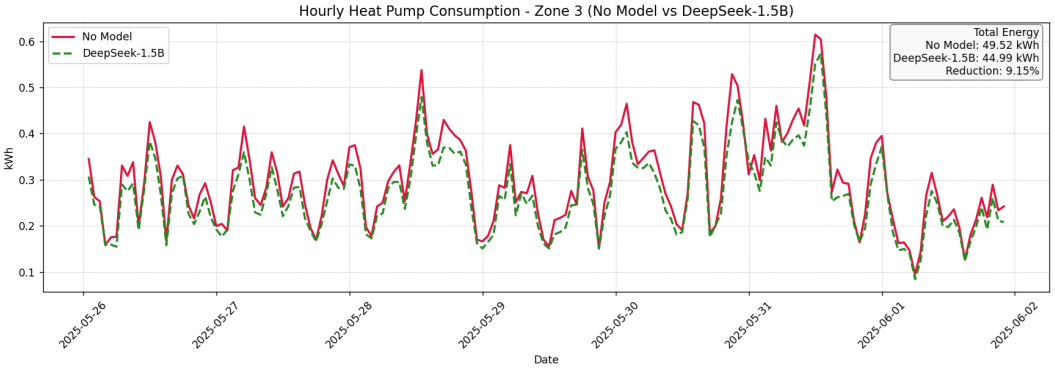


Figure 9. Zone 3 Thessaloniki pilot.

4.7. From Fine-Tuning to Deployment: Evaluating GPT-4's Operational Costs in Real-Time Smart Building Applications

The fine-tuning process for GPT-4o-2024-08-06 was evaluated under two configurations. In the initial experiment, a small dataset of 10 cases was used, with a batch size of 2 over 3 epochs, resulting in a training cost of approximately USD 0.21. Each inference query cost USD 0.01. Although these figures appear minimal in isolation, the true cost becomes significant during deployment: generating a recommendation every 15 min results in a daily cost of USD 0.96 per user, or, roughly, USD 350 annually per user. Scaling this to 100 users

leads to an annual operational cost of USD 35,000, highlighting the substantial financial overhead of using GPT-4 for real-time smart building applications, particularly when high-frequency recommendations are required. These findings are consistent with [19], which reported GPT-4 operational costs ranging from USD 0.30 to USD 1.00 per user per day, with response times reaching up to 45 s. A subsequent experiment expanded the dataset to 120 cases, using a batch size of 4 over 5 epochs. This larger setup increased the training cost to approximately USD 4, while the inference query cost remained unchanged at USD 0.01 per request. Although training costs remained modest, inference costs continued to represent the primary challenge, underscoring the importance of balancing model performance with long-term operational sustainability. These findings underscore the importance of balancing model performance with operational sustainability, particularly in real-time smart building applications where inference frequency is high. Selecting models that offer an optimal trade-off between accuracy and cost is therefore critical to ensuring scalability and long-term feasibility.

4.8. Model Selection Guidance

Beyond performance metrics, model selection in real-world smart building deployments must consider cost, latency, privacy, and integration constraints. While larger models such as DeepSeek-R1-Distill-Qwen-7B and GPT-4 consistently achieved higher ZASR and F1-scores in our experiments, smaller models like GPT-2-Small and GPT-2-Medium produced satisfactory outputs at significantly lower computational cost. To support practical adoption, we propose the following deployment-oriented guidance for selecting an appropriate model:

- **Resource-Constrained Environments:** GPT-2-Small and GPT-2-Medium are well suited for embedded systems, edge devices, or low-power local deployments, where hardware limitations and inference latency are critical.
- **Cloud-Based Decision Support:** DeepSeek-R1-Distill-Qwen-7B and GPT-4 are preferable in centralized setups where bandwidth, inference cost, and GPU availability are less restrictive, and where higher-quality language output is prioritized.
- **Data Sensitivity and Privacy:** In use cases where building telemetry includes sensitive occupant behavior or energy usage data, smaller models deployed locally offer enhanced data control.
- **High-Frequency Applications:** When generating frequent recommendations (e.g., every 15 min), inference cost becomes a dominant factor. In such cases, open-access models like GPT-2-Medium or DeepSeek-1.5B offer the best cost-efficiency balance.
- **Mission-Critical Scenarios:** In control-critical or safety-sensitive applications, where misinterpretations can lead to discomfort or energy loss, larger models may be justified, due to their superior semantic precision and generalization.

This informal decision framework allows practitioners to tailor model selection based on deployment constraints and performance expectations, bridging the gap between simulation results and operational integration. This informal framework allows stakeholders to make trade-offs based on operational needs, infrastructure availability, and acceptable inference cost.

5. Conclusions and Future Work

The findings of this article demonstrate that open-access models such as DeepSeek-R1-Distill-Qwen-1.5B, DeepSeek-R1-Distill-Qwen-7B, and GPT-2-Medium, when adequately fine-tuned, can deliver high-quality, context-aware recommendations that match or even surpass the performance of state-of-the-art commercial models like GPT-4. Although GPT-4 showed strong performance, even with relatively small training datasets, its high compu-

tational and financial cost poses significant limitations for large-scale or high-frequency deployment. In contrast, the open-access models offer a practical and cost-effective alternative for real-world smart building applications, particularly where scalability, frequent retraining, and operational sustainability are critical. Overall, the experimental results highlight the potential of smaller and mid-scale LLMs to achieve competitive performance while offering substantial advantages in computational efficiency and deployment cost. This underscores the importance of prioritizing open-access and resource-efficient solutions when designing scalable smart building systems. From a deployment perspective, the LLM-based recommendation system can be integrated into existing BMS architectures via RESTful APIs or MQTT interfaces, acting as a decision-support layer without replacing core control logic. Real-time building telemetry, such as indoor temperature, HVAC status, or occupancy, can be forwarded to the LLM, which returns human-readable recommendations or alerts. Lightweight models (e.g., GPT-2-Medium, DeepSeek-1.5B) may be deployed on local servers to ensure data privacy and low-latency inference, while heavier models like GPT-4 may be reserved for cloud-based analysis or offline retraining cycles.

Building on these findings, the practical considerations for integrating LLM-based systems into real-world Building Management Systems (BMSs) were also explored. From a deployment perspective, the proposed recommendation engine can be embedded into existing BMS architectures using RESTful APIs or MQTT interfaces, operating as a decision-support layer rather than replacing core control logic. Real-time telemetry data, such as indoor temperature, HVAC status, and occupancy, can be continuously streamed to the LLM, which returns human-readable recommendations to inform facility managers or downstream automation tools. Lightweight models (e.g., GPT-2-Medium, DeepSeek-1.5B) are particularly well-suited for local deployment scenarios where data privacy and response latency are critical, while heavier models like GPT-4 can be reserved for cloud-based analytics or batch processing tasks. To ensure long-term system reliability and adaptability, incorporating a retraining strategy tailored to the deployment environment is recommended. For smaller models, such as GPT-2-Small and GPT-2-Medium, retraining every one to three months may be advisable, particularly in environments with high variability in occupancy or seasonal energy usage. Larger models like DeepSeek-7B or GPT-4 typically generalize more effectively and may only require semi-annual fine-tuning. Furthermore, automated monitoring using domain-specific metrics such as ZASR can enable dynamic retraining triggers based on observed performance drops, thereby reducing unnecessary computational overhead while maintaining recommendation quality.

Looking forward, while the current system focuses on generating natural language outputs, future work will advance toward full automation by integrating direct control signals into Building Management Systems (BMSs). Additionally, the incorporation of Reinforcement Learning (RL) is planned to develop hybrid architectures in which LLMs are complemented by RL agents capable of continuous learning and adaptation in dynamic environments. Such integration will enhance decision-making flexibility, allowing buildings not only to recommend but also autonomously execute optimized strategies in response to evolving conditions, further advancing the vision of intelligent, self-regulating smart buildings. Another important direction involves validating the Zone-Aware Semantic Reward (ZASR) against human expert evaluations. While ZASR is designed to complement standard metrics like BERTScore by explicitly penalizing semantically valid but zone-inaccurate outputs, the absence of human-annotated validation is acknowledged in the current study. Future work will include expert-labeled datasets and use ZASR within Reinforcement Learning with Human Feedback (RLHF) pipelines to quantify its alignment with human-perceived quality and enable more targeted fine-tuning of recommendation outputs.

Author Contributions: Conceptualization, I.P. and C.K.; methodology, I.P., C.K., and E.K.; software, I.P. and C.K.; validation, I.P., E.K., and C.K.; formal analysis, C.K. and E.K.; investigation, I.P. and C.K.; resources, E.K.; data curation, C.K.; writing—original draft, I.P. and C.K.; writing—review and editing, I.P., C.K., and E.K.; visualization, E.K.; supervision, C.K.; project administration, C.K. All authors have read and agreed to the published version of the manuscript.

Funding: We acknowledge partial support of this work by the European Commission Horizon Europe-Harmonise: Hierarchical and Agile Resource Management Optimization for Networks in Smart Energy Communities (Grant agreement ID: 101138595).

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

Nomenclature

The following nomenclature are used in this manuscript:

BMS	Building Management System
EMS	Energy Management System
HVAC	Heating, Ventilation, and Air Conditioning
LLM	Large Language Model
ZASR	Zone-Aware Semantic Reward
F1-Score	Harmonic mean of precision and recall
PV	Photovoltaic
RL	Reinforcement Learning
ESS	Energy Storage System
EV	Electric Vehicle
ANN	Artificial Neural Network
GPT-2, GPT-4	Generative Pre-trained Transformer models
Sentence-BERT	Sentence-level Bidirectional Encoder Representations from Transformers
Sinergym	Building simulation environment built on EnergyPlus and Gym
DeepSeek-R1-Distill-Qwen-1.5B/7B	Open-access language models used for training and evaluation

References

1. Chamari, L.; Petrova, E.; Pauwels, P. An end-to-end implementation of a service-oriented architecture for data-driven smart buildings. *IEEE Access* **2023**, *11*, 117261–117281. [CrossRef]
2. Mercurio, A.; Di Giorgio, A.; Quaresima, A. Distributed control approach for community energy management systems. In Proceedings of the 2012 20th Mediterranean Conference on Control & Automation (MED), Barcelona, Spain, 3–6 July 2012; pp. 1265–1271.
3. Athanasiadis, C.L.; Pippi, K.D.; Papadopoulos, T.A.; Korkas, C.; Tsaknakis, C.; Alexopoulou, V.; Nikolaidis, V.; Kosmatopoulos, E. A smart energy management system for elderly households. In Proceedings of the 2022 57th International Universities Power Engineering Conference (UPEC), Istanbul, Turkey, 30 August–2 September 2022; pp. 1–6.
4. Michailidis, I.T.; Korkas, C.; Kosmatopoulos, E.B.; Nassie, E. Automated control calibration exploiting exogenous environment energy: An Israeli commercial building case study. *Energy Build.* **2016**, *128*, 473–483. [CrossRef]
5. Fanti, M.P.; Mangini, A.M.; Roccotelli, M. A petri net model for a building energy management system based on a demand response approach. In Proceedings of the 22nd Mediterranean Conference on Control and Automation, Palermo, Italy, 16–19 June 2014; pp. 816–821.
6. Korkas, C.D.; Terzopoulos, M.; Tsaknakis, C.; Kosmatopoulos, E.B. Nearly optimal demand side management for energy, thermal, EV and storage loads: An Approximate Dynamic Programming approach for smarter buildings. *Energy Build.* **2022**, *255*, 111676. [CrossRef]
7. Papaioannou, A.; Dimara, A.; Krinidis, S.; Anagnostopoulos, C.N.; Ioannidis, D.; Tzovaras, D. Advanced proactive anomaly detection in multi-pattern home appliances for energy optimization. *Internet Things* **2024**, *26*, 101175. [CrossRef]

8. Papaioannou, I.; Dimara, A.; Korkas, C.; Michailidis, I.; Papaioannou, A.; Anagnostopoulos, C.N.; Kosmatopoulos, E.; Krinidis, S.; Tzovaras, D. An Applied Framework for Smarter Buildings Exploiting a Self-Adapted Advantage Weighted Actor-Critic. *Energies* **2024**, *17*, 616. [CrossRef]
9. Jiménez-Raboso, J.; Campoy-Nieves, A.; Manjavacas-Lucas, A.; Gómez-Romero, J.; Molina-Solana, M. Sinergym: A building simulation and control framework for training reinforcement learning agents. In Proceedings of the 8th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation, Coimbra, Portugal, 17–18 November 2021; pp. 319–323.
10. Zhang, L.; Chen, Z. Opportunities and Challenges of Applying Large Language Models in Building Energy Efficiency and Decarbonization Studies: An Exploratory Overview. *arXiv* **2023**, arXiv:2312.11701.
11. Jiang, G.; Ma, Z.; Zhang, L.; Chen, J. EPlus-LLM: A large language model-based computing platform for automated building energy modeling. *Appl. Energy* **2024**, *367*, 123431. [CrossRef]
12. Jiang, G.; Ma, Z.; Zhang, L.; Chen, J. Prompt engineering to inform large language model in automated building energy modeling. *Energy* **2025**, *316*, 134548. [CrossRef]
13. Liu, H.; Suzuki, S.; Hyodo, A. LLMs Based Multi-Modal Location Recommendation for Smart Building. In *Electronics, Communications and Networks*; IOS Press: Amsterdam, The Netherlands, 2024; pp. 72–77.
14. Mshragi, M.; Petri, I. Fast machine learning for building management systems. *Artif. Intell. Rev.* **2025**, *58*, 211. [CrossRef]
15. Reimers, N.; Gurevych, I. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv* **2019**, arXiv:1908.10084.
16. Zhang, T.; Kishore, V.; Wu, F.; Weinberger, K.Q.; Artzi, Y. Bertscore: Evaluating text generation with bert. *arXiv* **2019**, arXiv:1904.09675.
17. Giudici, M.; Padalino, L.; Paolino, G.; Paratici, I.; Pascu, A.I.; Garzotto, F. Designing Home Automation Routines Using an LLM-Based Chatbot. *Designs* **2024**, *8*, 43. [CrossRef]
18. Rey-Jouanchicot, J.; Bottaro, A.; Campo, E.; Bouraoui, J.L.; Vigouroux, N.; Vella, F. Leveraging Large Language Models for enhanced personalised user experience in Smart Homes. *arXiv* **2024**, arXiv:2407.12024.
19. King, E.; Yu, H.; Lee, S.; Julien, C. Sasha: Creative goal-oriented reasoning in smart homes with large language models. In *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*; ACM: New York, NY, USA, 2024; Volume 8, pp. 1–38.
20. Trichopoulos, G.; Konstantakis, M.; Alexandridis, G.; Caridakis, G. Large language models as recommendation Systems in Museums. *Electronics* **2023**, *12*, 3829. [CrossRef]
21. Ahn, K.U.; Kim, D.W.; Cho, H.M.; Chae, C.U. Alternative Approaches to HVAC Control of Chat Generative Pre-Trained Transformer (ChatGPT) for Autonomous Building System Operations. *Buildings* **2023**, *13*, 2680. [CrossRef]
22. Dedeoglu, V.; Zhang, Q.; Li, Y.; Liu, J.; Sethuvenkatraman, S. BuildingSage: A safe and secure AI copilot for smart buildings. In Proceedings of the 11th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation, Hangzhou, China, 7–8 November 2024; pp. 369–374.
23. Kök, İ.; Demirci, O.; Özdemir, S. When IoT Meet LLMs: Applications and Challenges. In Proceedings of the 2024 IEEE International Conference on Big Data (BigData), Washington DC, USA, 15–18 December 2024; pp. 7075–7084.
24. Shahrabani, M.M.N.; Apanaviciene, R. An AI-Based Evaluation Framework for Smart Building Integration into Smart City. *Sustainability* **2024**, *16*, 8032. [CrossRef]
25. Oprea, S.V.; Bâra, A. A Recommendation System for Prosumers Based on Large Language Models. *Sensors* **2024**, *24*, 3530. [CrossRef] [PubMed]
26. Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; Sutskever, I. Language models are unsupervised multitask learners. *OpenAI Blog* **2019**, *1*, 9.
27. Guo, D.; Yang, D.; Zhang, H.; Song, J.; Zhang, R.; Xu, R.; Zhu, Q.; Ma, S.; Wang, P.; Bi, X.; et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv* **2025**, arXiv:2501.12948.
28. Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. Gpt-4 technical report. *arXiv* **2023**, arXiv:2303.08774.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.